OJT Project Design Template

Student Name(s): Piyush Thakkar and Archi Tyagi
Roll No(s): 251810700055 and 251810700303
Year & Section: 2025 and 1A
Project Title (as assigned): Electricity Consumption Patterns
Project Type: Data Scientist
Stack / Framework:

Frontend: React.js

Backend: Python (FastAPI)

Database: PostgreSQL (with TimescaleDB extension for time-series optimization)

Data Processing: Pandas, NumPy

Visualization: Matplotlib, Seaborn

Analytics: Time-Series Analysis Libraries (statsmodels, prophet)

Machine Learning: Forecasting Models (scikit-learn, ARIMA, LSTM)

Deployment (Optional): Docker

Additional Libraries: - **Backend:** SQLAlchemy (ORM), PyJWT (authentication), Uvicorn (ASGI server), Pydantic (validation), SciPy (statistical analysis) - **Frontend:** TailwindCSS (styling), Recharts/Plotly.js (interactive charts), Axios (API calls), React Query (data fetching) - **Development Tools:** Git/GitHub, Postman (API testing), pytest (testing)

---

1. Problem Understanding

1.1 What is the problem statement in your own words?

India is installing 250 million smart meters by 2027 that collect detailed electricity usage data every 15 minutes. But this valuable data is not being used properly. Consumers get monthly bills without knowing which appliances use the most electricity or when they should reduce usage to save money. This is especially important in India because electricity rates increase as you use more units - crossing into a higher slab can make your bill much more expensive. Electricity companies (DISCOMs) don’t have tools to predict demand in real-time, which causes power cuts during peak hours and makes the grid unstable. Building managers can’t figure out which equipment wastes energy or how to split costs fairly among different floors or tenants. The core problem is simple: huge amounts of consumption data is being collected from smart meters, but there’s no easy-to-use system that turns this data into useful insights. This means consumers can’t reduce their bills, utilities can’t manage the grid better, and building managers can’t optimize their operations.

1.2 Why does this problem exist or matter?

This problem matters because it affects millions of people and wastes billions of rupees in India’s energy sector. **Consumers** benefit by understanding their usage patterns and reducing electricity bills - even a 10-15% reduction means saving ₹60-150/month for average households (₹700-1,800/year). **DISCOMs and utilities** benefit by reducing the 19.2% AT&C losses (theft and technical losses), better forecasting demand to prevent blackouts, and implementing time-of-use pricing that balances the grid. **Building managers** benefit by identifying inefficient equipment, fairly allocating costs, and reducing operational expenses. **The environment** benefits as better insights lead to reduced energy waste and lower carbon emissions.

Developers and data scientists benefit from working with real-world time-series data and building impactful analytics systems. **The broader Indian community** benefits from a more stable power grid, reduced power cuts, and progress toward India's energy efficiency and renewable energy goals. With India's smart meter investment of ₹1.25 lakh crores, solving this problem can unlock ₹9.5 lakh crores in potential savings by eliminating losses and improving grid efficiency.

1.3 Key inputs and expected outputs:

Inputs	Process	Expected Outputs
Smart meter consumption data (15-min intervals, kWh readings, timestamps)	Data ingestion pipeline, validation, time-series storage in PostgreSQL with TimescaleDB extension	Clean, structured consumption database with time-based partitioning
User details (location, state, DISCOM, connection type)	User authentication, role-based access control, tariff mapping	Personalized user profiles with correct tariff structure
Tariff structures from SERCs (slab rates, fixed charges, ToU pricing)	Tariff database management, state-wise configuration	Accurate bill calculations based on user's state/DISCOM
Historical consumption patterns	Time-series analysis (trend, seasonality, patterns), statistical computations (mean, peak, off-peak) using Pandas/NumPy	Consumption insights: daily/weekly/monthly patterns, peak hours identification
Real-time consumption data	Real-time processing, slab monitoring, cost calculation	Live dashboard showing current usage, cost tracker, slab alerts
Multiple households' data (anonymized)	Aggregation, comparison algorithms, benchmarking	Comparative analytics: how user compares to similar households
Appliance power ratings, usage duration	Appliance-level breakdown estimation, cost attribution	Appliance-wise consumption and cost breakdown
ML model predictions	Integration with forecasting models, demand prediction	Future consumption forecasts, bill predictions, optimization recommendations

---

2. Functional Scope

2.1 What are the core features you plan to build (must-haves)?

1. User Authentication & Profile Management - **Backend:** User registration/login APIs with JWT authentication, profile storage in PostgreSQL - **Frontend:** Registration and login forms, profile setup wizard (location, state, DISCOM selection)

2. Data Ingestion & Real-Time Dashboard - **Backend:** Load dataset, replay with current timestamps via WebSocket, validate and store in PostgreSQL with TimescaleDB - **Frontend:** Live-updating

dashboard showing current consumption (kWh), real-time cost tracker, animated meters and progress bars

3. Consumption Analytics & Visualizations - **Backend:** RESTful APIs for consumption data (hourly/daily/weekly/monthly), statistical analysis (mean, peak, off-peak), pattern detection - **Frontend:** Interactive charts (line graphs, bar charts, heatmaps), date range selectors, filterable views

4. Bill Calculation & Cost Tracking - **Backend:** Slab-based tariff calculation engine for all Indian states, real-time cost computation, bill estimation APIs - **Frontend:** Bill breakdown display (slab-wise costs), month-to-date tracker, alerts when approaching higher slabs

5. Comparison & Benchmarking - **Backend:** Aggregation queries for similar households, state/national averages, anomaly detection algorithms - **Frontend:** Comparison charts (user vs neighbors vs state average), consumption ranking, efficiency score

6. Gamification & Social Features - **Backend:** Leaderboard system (neighborhood/city/state rankings), achievement engine, challenge tracking, points calculation - **Frontend:** Energy savings leaderboard, badges and achievements, progress bars, weekly challenges

2.2 What stretch goals could you attempt if time permits?

1. AI-Powered Recommendations & Forecasting - **Backend:** ML models for consumption prediction, personalized cost-saving recommendations, appliance-level breakdown estimation - **Frontend:** Smart suggestions cards, future bill predictions with confidence intervals, “What-if” scenarios

2. IoT & Smart Home Integration - **Backend:** Integration with smart plugs/IoT devices, real-time appliance monitoring, automated control APIs - **Frontend:** Appliance control dashboard, automated scheduling interface, energy-saving automation rules

3. Utility-Grade Features for DISCOMs - **Backend:** Grid load forecasting, demand response program management, outage detection, theft/tamper detection algorithms - **Frontend:** Utility operator dashboard, grid heatmaps, real-time demand monitoring, alert management system

4. Renewable Energy Integration - **Backend:** Solar panel production tracking, net metering calculations, battery storage management - **Frontend:** Solar production vs consumption graphs, savings display, ROI calculator

5. Community Sharing & Social Networks - **Backend:** Social sharing APIs, friend system, group challenges, public profile management - **Frontend:** Share achievements on social media, create private groups, team competitions

2.3 Which libraries or tools will you use?

Frontend: React.js

Backend: Python (FastAPI)

Database: PostgreSQL (with TimescaleDB extension for time-series optimization)

Data Processing: Pandas, NumPy

Visualization: Matplotlib, Seaborn

Analytics: Time-Series Analysis Libraries (statsmodels, prophet)

Machine Learning: Forecasting Models (scikit-learn, ARIMA, LSTM)

Deployment (Optional): Docker

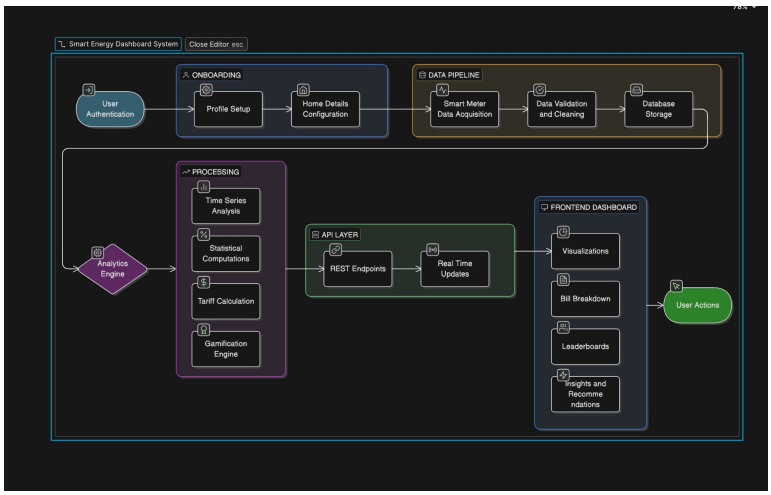
Additional Libraries: - **Backend:** SQLAlchemy (ORM), PyJWT (authentication), Uvicorn (ASGI server), Pydantic (validation), SciPy (statistical analysis) - **Frontend:** TailwindCSS (styling), Recharts/Plotly.js (interactive charts), Axios (API calls), React Query (data fetching) - **Development Tools:** Git/GitHub, Postman (API testing), pytest (testing)

---

3. System & Design Thinking

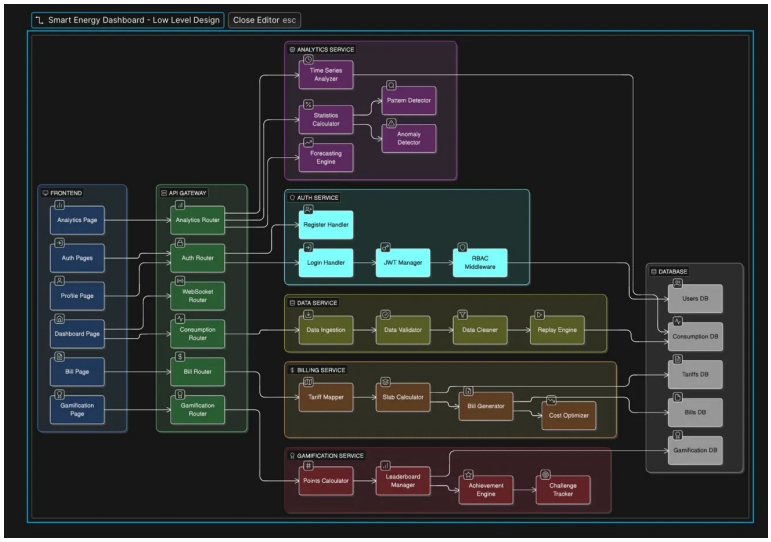
3.1 Sketch or describe your app flow / pipeline:

High Level Design (HLD):



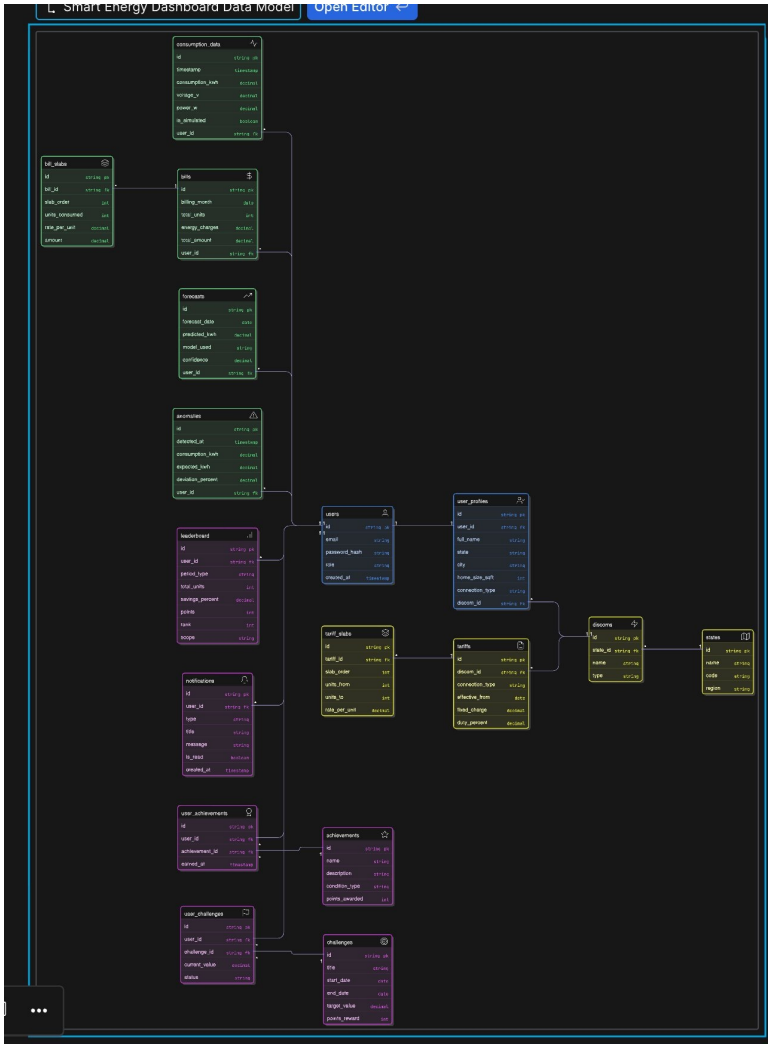
High Level Design

Low Level Design (LLD) - Component Diagram:



Low Level Design

Low Level Design (LLD) - Database Schema (ER Diagram):



ER Diagram

Simplified Flow:

```
User Login → Data Ingestion → Analytics Processing →
Bill Calculation → Visualization → User Insights & Actions
```

3.2 What data structures or algorithms are central to this project?

Data Structures: - **Arrays** - Store consumption readings (e.g., [2.5, 3.1, 2.8, ...] kWh recorded every 15 minutes) - **Dictionaries/Hash Maps** - User info, tariff rates for different states - **Database Tables** - PostgreSQL tables for users, consumption history, bills, leaderboards - **Sorted Lists** - Leaderboards (top savers)

Algorithms: - **Bill Calculation** - Loop through consumption slabs - **Averages & Statistics** - Calculate daily/weekly/monthly averages, find peak hours - **Grouping/Aggregation** - Sum 15-min readings into hourly/daily/monthly totals - **Sorting & Ranking** - Rank users for leaderboards - **Pattern Detection** - Find peak consumption hours, detect unusual spikes - **Time-Series Analysis** - Identify consumption trends over weeks/months

3.3 How will you test correctness or performance?

Testing Strategy:

1. Functional Testing - Manual testing of all core features (authentication, dashboard, visualizations, bill calculation, leaderboard) - Verify user flows: signup → profile setup → view consumption → compare with others - Test on different screen sizes (desktop, tablet, mobile)

2. Unit Testing (pytest) - Test critical functions: bill calculation logic, tariff slab computations - Verify data validation (valid timestamps, consumption values within realistic ranges) - Test API

endpoints with sample inputs

3. Data Accuracy Verification - Validate bill calculations against manual calculations for different consumption levels - Cross-check state-specific tariff rates with official GERC/SERC tariff orders - Verify statistical computations (averages, peaks) against expected values

4. Performance Checks - Monitor dashboard load time (target: <3 seconds) - Check API response times for common queries - Verify smooth real-time updates via WebSocket

Success Metrics: - ✅ All core features working without crashes - ✅ Bill calculation matches manual calculations - ✅ Dashboard displays data correctly - ✅ Real-time updates visible and smooth - ✅ No major bugs during demo

---

4. Timeline & Milestones (12 Weeks)

Week	Planned Deliverables	Mentor Checkpoint
W1	Domain research, requirement analysis, dataset exploration (IIT Bombay/iAWE)	☐
W2	Environment setup, database schema design, project architecture	☐
W3	Data preprocessing, cleaning, loading dataset into PostgreSQL	☐
W4	Backend: User authentication, JWT, profile management APIs	☐
W5	Backend: Data ingestion pipeline, TimescaleDB optimization, WebSocket setup	☐
W6	Backend: Bill calculation engine, tariff logic for all Indian states	☐
W7	Backend: Analytics APIs (consumption patterns, statistics, comparisons)	☐
W8	Backend: Gamification engine (leaderboards, achievements, challenges)	☐
W9	Frontend: Dashboard, real-time consumption charts, visualizations	☐
W10	Frontend: Bill breakdown, comparison charts, leaderboard UI	☐
W11	Integration, testing, bug fixes, performance optimization	☐
W12	Final testing, deployment, demo preparation, documentation	☐

---

5. Risks & Dependencies

5.1 What's the hardest part technically for you right now?

The most challenging aspects are:

1. Understanding and implementing time-series data processing with PostgreSQL/TimescaleDB (new technology for us)
2. Building slab-based tariff calculation logic that works correctly for all Indian states
3. Implementing real-time WebSocket updates and ensuring smooth frontend-backend integration
4. Designing an engaging gamification system (leaderboards, badges, challenges) that motivates users
5. Managing the scope - we have ambitious features and need to prioritize what's achievable in 12 weeks

5.2 What dependencies or help do you need from mentors?

Mentor support needed:

- Guidance on PostgreSQL/TimescaleDB setup and optimization for time-series data
- Review of tariff calculation logic to ensure accuracy
- Feedback on system architecture and database schema design
- Help with WebSocket implementation and real-time data streaming
- Code review for backend APIs and frontend integration
- Suggestions for prioritizing features if timeline becomes tight

---

6. Evaluation Readiness

6.1 How will you prove that your project "works"?

Deliverables to demonstrate working project:

- Live demo of the web application (deployed link)
- GitHub repository with complete source code
- Demo video showing: user signup → dashboard → real-time updates → bill calculation → leaderboard
- Screenshots of key features (authentication, analytics charts, bill breakdown, gamification)
- Presentation slides explaining system architecture and features

6.2 What success metric or goal will you aim for?

Success Metrics:

- 100% functional core features
- Accurate bill calculations
- Smooth real-time dashboard updates
- Working visualizations and gamification
- No critical bugs during demo

---

7. Responsibilities

Task	Piyush Thakkar	Archi Tyagi	Mentor Notes
Backend Development (FastAPI, PostgreSQL, APIs, Authentication)	☑	☐	
Frontend Development (React, Dashboard, Charts, UI/UX)	☐	☑	
Data Processing & Analytics (Pandas, Time-	☑	☐	

series, Bill Calculation)			
Gamification & Features (Leaderboard, Badges, Comparison)	☐	☑	
Testing, Deployment & Documentation	☑	☑	Both collaborate

Students:
Piyush Thakkar (251810700055)
Archi Tyagi (251810700303)

Mentor Approval: _____

Date: February 15, 2026